Stockholm Public Transport Delay Analytics Using Apache Spark

Group 67: Hieu Vu, Rong Jin, Serban

1 Goal and dataset

We analyse one month of real-time data from Stockholm's public transport (SL) to answer four questions: which lines have the highest average delays, which stations are most often delayed, during which hours delays are worst, and how weekdays and weekends differ.

The data comes from Trafiklab's **KoDa** archive of SL's open feeds (`trafiklab.se/api/trafiklab-apis/koda`, free API key; too large to include, but `scripts/download_koda.py` downloads it). We use two parts: **GTFS-RT TripUpdates**, a snapshot of every running trip's arrival times and delays taken about every 14 seconds (around 6,000 files and 4.5 GB unpacked per day), and the **static GTFS** timetable valid on each day, which turns trip and stop ids into line numbers, station names, coordinates and scheduled times. We downloaded 7–8 and 12–27 September 2026. Days with both parts give **16 service days** (12 weekdays, 4 weekend days): **9.5 million vehicle stops** on 481,507 trips, 578 lines and 5,891 stations.

2 Methodology

Storage and tools. A single-node HDFS (Hadoop 3.4.3) holds all data as Parquet, partitioned by day (and by hour for the raw delays); processing is PySpark 4; the dashboard is Plotly Dash.

Ingestion (`hdfs-ingression.py`). Timetable tables are converted from CSV to Parquet, one copy per day. The trip update snapshots are parsed from protobuf; since each snapshot repeats every trip with a revised estimate, only the newest row per trip, service day and stop is kept within each hour (about 96× fewer rows, with no information lost). A `--day-by-day` mode unpacks, ingests and deletes one date at a time, so the 85 GB month fits on a laptop.

Preprocessing (Spark, `spark-preprocessing.py`). (1) A window keeps the newest row per (trip, stop, service day) across all hour folders, as one stop appears in several hours while its trip runs; the service day is part of the key because SL reuses trip ids. (2) Stops whose time was still in the future when last seen never happened (e.g. cancelled) and are dropped. (3) The delay is the departure delay at a trip's first stop (vehicles arrive early and wait) and the arrival delay elsewhere; missing values and delays outside −15 min to +2 h are removed. (4) Each stop is joined with the *same day's* timetable (trips, routes, stops, stop_times) and platforms are mapped to their parent station. (5) We add the scheduled Stockholm hour, weekday/weekend, transport mode and a *late* flag ($\geq$3 minutes). 15.3 M hourly rows become 10.1 M distinct stops and 9.5 M joined stop events; the rows lost in the join belong almost entirely to days whose timetable was not downloaded.

Analytics (Spark, `spark-analytics.py`). For lines, stations, hours, day types, transport modes and one-minute delay bins we compute the number of stops, mean, exact median and 90th percentile delay, and the share of late stops. Lines are grouped by `route_id` and stations by `station_id`, not by name (line "14" is both a metro and a ferry; "Slussen" is five stations). Only lines with $\geq$100 and stations with $\geq$50 stops per day are ranked. Results are written to HDFS and exported as CSV for the dashboard (`dashboard.py`).

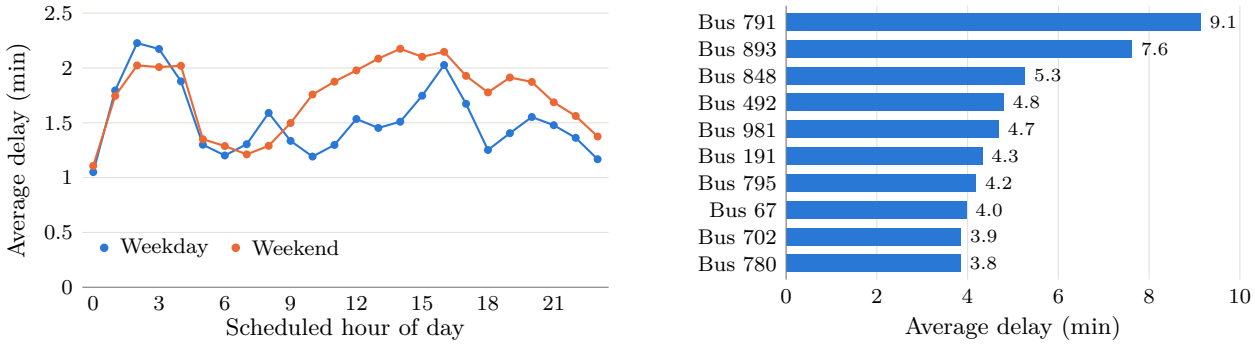
Verification. Both Spark stages were reimplemented independently in pandas and compared value by value (all rows and columns identical), and sampled stops were traced back to the raw protobuf files. This caught real bugs, e.g. a station split in two after its coordinates moved between timetables.

3 Results

On average a stop is **92 s late** (median 60 s) and **20.3% of stops are 3+ minutes late**; a quarter of stops are early and 1.5% are 10+ minutes late.

Lines (Fig. 1b). The ten most delayed lines are all buses: bus 791 averages 9.1 min late and is late at 85% of its stops, followed by buses 893 (7.6 min) and 848 (5.3 min). Rail is far more punctual: no metro, commuter train or tram line averages more than about a minute late, against a median of 88 s across bus lines.

Stations. The stations most often 3+ minutes late are bus stops in two corridors: around Tungelsta (Tungelsta skola 76.5%, Mulstavägen 66.8%), served by buses 835, 848 and 893, and in Östermalm on bus 67 (Skeppargatan, Runebergsplan, Historiska museet, Strandvägen, all around 60–62%). Delays accumulate along a trip, so a station's punctuality mostly reflects the lines through it. The busiest stations by vehicle stops are bus terminals



(a) Average delay by hour, all modes.

(b) The ten lines with the highest average delay.

Figure 1: Delays by hour and by line, 16 service days in September 2026.

(Londonviadukten 2,299 stops/day, Henriksdal 2,199) and T-Centralen (1,868, all seven metro lines); being busy does not imply being late (Henriksdal 35% late, T-Centralen 2%).

Hours (Fig. 1a). On weekdays delays build through the day and peak in the afternoon rush at 16:00 (2.0 min, 27.6% late), with a smaller morning peak at 08:00; 06:00 and 10:00 are the most punctual daytime hours. The high values between 01:00 and 04:00 come from only 2.8% of all stops.

Weekday vs weekend. Weekends are worse overall (108 s vs 89 s average; 23.2% vs 19.7% late), but only because of buses, which make up 87% of stops: every other mode is *more* punctual at the weekend (Table 1). The weekend gap is largest between 10:00 and 14:00, and the two Saturdays were the worst days of the month.

Table 1: Average delay and share of late stops by transport mode.

	Average delay		Late (≥3 min)	
Mode	Weekday	Weekend	Weekday	Weekend
Bus	98 s	124 s	21.8%	26.8%
Ferry	106 s	86 s	24.0%	20.3%
Tram	29 s	24 s	3.3%	3.0%
Train	26 s	−5 s	8.5%	3.5%
Metro	13 s	3 s	2.5%	0.9%
All	89 s	108 s	19.7%	23.2%

Limitations. Sixteen September days (two weekends, no holidays); every vehicle stop counts the same regardless of passengers; cancelled trips have no actual time and are not counted as delays; and the pipeline runs on a single machine, although it is written for a cluster.

4 Running the code

Requirements: Python 3.10+, Java 17 and Hadoop 3.4 (single-node HDFS; setup steps in `README.md`). Put a KoDa API key in `.env` as `KODA_KEY` (see `.env.example`).

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
hdfs --daemon start namenode && hdfs --daemon start datanode
python scripts/download_koda.py --start 2026-09-07 --end 2026-09-27  # to data/raw
python scripts/hdfs-ingression.py --day-by-day   # Parquet into HDFS /sl
python scripts/spark-preprocessing.py            # /sl/stop_events
python scripts/spark-analytics.py                # /sl/analytics, data/analytics
python scripts/dashboard.py                      # http://127.0.0.1:8050
```